
LnT Camp 2026

Bridging the Gap: Empowering Future Talent through Machine Learning for Industry Innovation

FINAL PROJECT GUIDELINE

3-WEEK EDITION

1. Overview

This document is the official guideline for the **Final Project of LnT Camp 2026**. It is designed for a **3-week working period** following the completion of all 10 sessions. The scope has been calibrated so that a team can deliver a complete, meaningful, and deployable ML solution within the time available.

The theme — *"Bridging the Gap: Empowering Future Talent through Machine Learning for Industry Innovation"* — should be reflected throughout your project's framing, insights, and presentation.

2. Team Composition

- Each team consists of 1 - 4 participants.
- If a team consists of more than one member, one member must be designated as the Team Leader who coordinates task division and submissions.
- All members are expected to contribute to the notebook, the codebase, and the LinkedIn post.

3. Dataset

3.1 Source

The dataset is derived from the **Global Superstore Dataset** (originally published on Kaggle: <https://www.kaggle.com/datasets/fatihilhan/global-superstore-dataset>). For this project, it has been restructured into a normalised SQLite database.

SQLite Database Download:

<https://drive.google.com/file/d/1M2sonY7serOCzWYDCKEdxZ5fj7quOoKd/view?usp=sharing>

3.2 Database Schema (ERD Summary)

The SQLite database contains the following tables:

ERD Diagram: The full Entity Relationship Diagram is available at <https://drive.google.com/file/d/1P7BqgEAtetiEq0hoZ8uQqYkslynnZMpYI/view?usp=sharing>. Teams are encouraged to study the relationships between tables before starting their data loading queries.

Table	Key Columns	Description
orders	order_key (PK), customer_id (FK), location_id (FK)	One record per order; contains order/ship dates, priority, and ship mode
order_items	row_id (PK), order_key (FK), product_id (FK)	Line items with sales, profit, discount, quantity, and shipping cost
dim_customers	customer_id (PK)	Customer name and segment (Consumer / Corporate / Home Office)

Table	Key Columns	Description
dim_products	product_id (PK)	Category, sub-category, and product name
dim_locations	location_id (PK)	City, state, country, region, and market
dim_product_name_variants	product_id (FK)	Raw product name variants for the same product_id

Relationships: *orders* links to *dim_customers* and *dim_locations* via foreign keys. *order_items* links to *orders* and *dim_products*. Use SQL JOIN queries to assemble your working DataFrame.

3.3 Suggested Target Variables

The following are starting points. Teams may explore other meaningful targets based on their EDA findings:

- Regression: Predict sales or profit for an order item.
- Classification: Predict order_priority or whether an order will be profitable (binary).
- Clustering: Segment customers by purchasing behaviour, or segment products by performance.

Feature engineering tip: Derived features such as profit margin, delivery duration (ship_date – order_date), or discount tier categories often improve model performance significantly and will be rewarded in the assessment.

4. Modelling Tasks

Team choice: Each team selects 2 of the 3 tasks below. This choice must be consistent throughout the notebook, the backend, and the frontend. Declare your chosen tasks clearly in the notebook's Problem Statement section.

Task	Goal	Required Metrics	Suggested Algorithms
Regression	Predict a continuous value (e.g. sales, profit, shipping cost)	MAE, RMSE, R ²	Linear Regression, Ridge/Lasso, Random Forest, XGBoost
Classification	Predict a category (e.g. order priority, profitable vs not)	Accuracy, Precision, Recall, F1, ROC-AUC	Logistic Regression, SVM, Decision Tree, Random Forest
Clustering	Discover natural groupings (e.g. customer segments, product tiers)	Silhouette Score, Elbow/Inertia	K-Means, DBSCAN, Agglomerative

4.1 Regression (if chosen)

- Goal: Predict a continuous numerical value (e.g. sales, profit, or shipping cost).
- Required metrics: MAE, RMSE, and R^2 .
- Implement one algorithm. Justify your choice briefly in a markdown cell.
- Interpret the results in business terms — not just numbers.

4.2 Classification (if chosen)

- Goal: Predict a categorical label (e.g. order priority, profitable vs non-profitable).
- Required metrics: Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- Implement one algorithm. Address class imbalance if present (e.g. class weights or oversampling).
- Interpret results — which class does the model struggle with, and why might that matter?

4.3 Clustering (if chosen)

- Goal: Discover natural groupings within the data (e.g. customer segments, product tiers).
- Required evaluation: Silhouette Score plus Elbow Method or Inertia if using K-Means.
- Implement one algorithm. Assign a meaningful descriptive label to each cluster.
- Explain what each cluster represents and how the business could act on it.

5. Deliverables

All deliverables must be submitted by the Final Submission Deadline. The table below summarises what is expected:

Deliverable	Contents	Notes
Jupyter Notebook (.ipynb)	Full ML pipeline: EDA, preprocessing, and both chosen modelling tasks	Every code cell must have a markdown explanation and output interpretation
GitHub Monorepo	model/, backend/, frontend/, notebook/ folders in one repository	One root README.md covering the whole project is sufficient
Backend API	REST API (FastAPI / Flask) with prediction endpoint(s) for chosen models	Must be callable from the frontend. Include setup instructions in README
Frontend App	Simulation page: user inputs values, frontend calls backend, result is displayed	Streamlit recommended. Must be publicly deployed. EDA dashboard is optional

Deliverable	Contents	Notes
LinkedIn Post	Project summary, GitHub link, deployed app link, all members tagged	Posted publicly by one member. Attach demo video, slides, or infographic.

5.1 Jupyter Notebook (.ipynb)

The notebook must be well-organised and include the following sections in order:

1. Problem Statement & Objectives (including declared modelling task choices)
2. Database Connection & Data Loading (using sqlite3 or sqlalchemy)
3. Exploratory Data Analysis (EDA)
4. Data Preprocessing & Feature Engineering
5. Modelling Task 1 — implementation, evaluation, and interpretation
6. Modelling Task 2 — implementation, evaluation, and interpretation
7. Conclusion & Business Insights

Important: Every code cell must be paired with a markdown cell explaining what the code does and interpreting the output. Raw code without explanation will result in a deduction.

5.2 GitHub Monorepo

All code lives in **one GitHub repository** with the following folder structure:

- notebook/ — Jupyter notebook and dataset access instructions
- model/ — saved model files (.pkl / .joblib) and any training scripts
- backend/ — REST API code and requirements file
- frontend/ — frontend application code

One **root-level README.md** covering the full project is sufficient. It should describe the project, list the two chosen modelling tasks, explain the folder structure, and provide setup/run instructions for both the backend and frontend.

5.3 Backend API

The backend exposes at least **one prediction endpoint per chosen modelling task**. The frontend must call the backend to display predictions — do not embed model logic directly in the frontend.

- Recommended: FastAPI or Flask.
- The API loads the saved model file from the model/ folder (no retraining on startup).
- Include a /health or /ping endpoint to confirm the service is running.
- Document the endpoints (input schema, output schema) in the README.

Deployment note: The backend does not need to be publicly deployed — it may run locally during the demo. However, if you deploy it (e.g. on Render or Railway), the frontend can be fully self-contained in the cloud.

5.4 Frontend Application

The frontend must include a **model simulation page**: a form where the user enters feature values, the frontend sends a request to the backend API, and the prediction result is displayed. One simulation page per chosen modelling task is expected.

- Recommended tool: Streamlit. Other frameworks (Dash, Flask+HTML, Next.js, etc.) are accepted.
- The app must be publicly deployed (e.g. Streamlit Community Cloud, Render, Vercel, Railway).
- An EDA dashboard page is optional — include it only if time allows.

Tip: Keep the UI simple. A clean form with labelled inputs and a visible prediction output is perfectly sufficient. Judges care about functionality, not visual polish.

5.5 LinkedIn Post

One team member must publish a LinkedIn post that:

- Summarises the project and key findings in language accessible to a general audience.
- States which two modelling tasks were chosen and what was discovered.
- Includes the link to the deployed frontend and the GitHub monorepo.
- Tags all team members by their LinkedIn profiles.
- Tags the speaker's account: <https://www.linkedin.com/in/yahyapp/>
- Attaches supporting media: a demo video of the app, slide deck, infographic, or screenshot.

Important: The post must be set to Public visibility. Submit the post URL as part of your final submission. Write for a non-technical reader — explain the project as you would to a friend outside the tech industry.

6. Assessment Criteria

Projects are assessed on the following rubric. The maximum score is 100 points.

Component	Description	Weight
Exploratory Data Analysis	Completeness, depth of insight, visualisation quality, and storytelling clarity	20%
Modelling (Task 1)	Correct implementation, appropriate metric reporting, and meaningful interpretation of results	15%
Modelling (Task 2)	Correct implementation, appropriate metric reporting, and meaningful interpretation of results	15%
Notebook Quality	Structure, markdown explanations, code cleanliness, and reproducibility	15%

Component	Description	Weight
GitHub (Monorepo)	Folder structure, README completeness, and code organisation	10%
Backend API	Working prediction endpoint(s), correct model loading, and code clarity	10%
Frontend Application	Model simulation UI, usability, and public deployment	10%
LinkedIn Post	Clarity, professionalism, audience reach, correct tagging, and attached media	5%
Total		100%

6.1 Grading Notes

- Modelling Tasks 1 & 2 are scored independently and equally. A weak performance on one task will not cancel out a strong performance on the other.
- Insight quality matters more than model accuracy. A well-interpreted result with modest performance beats a high-score model with no explanation.
- Notebook structure and markdown quality are assessed as a standalone component — they are not implied by model correctness.
- The LinkedIn post carries 5% — it is low weight but not optional. A missing post will lose all 5 points.
- Incomplete deliverables (missing repo folder, missing endpoint, missing post) result in a zero for that component.

7. Timeline & Deadlines

The project runs for roughly three weeks: **10 September 2026 until 4 October 2026 at midnight.**

Week	Period	Focus & Milestones
Week 1	10 September 2026 - 16 September 2026	Database connection · Data loading · EDA · Preprocessing & feature engineering
Week 2	17 September 2026 - 23 September 2026	Build & evaluate both chosen models · Finalize notebook writeup
Week 3	24 September 2026 - 3 October 2026	Backend API · Frontend deployment · LinkedIn post · Final submission

Week	Period	Focus & Milestones
Submission	4 October 2026; 23.59	All deliverables due: notebook, monorepo, deployed frontend, LinkedIn post link

Recommended week-by-week task split:

- Week 1: EDA & preprocessing; Setting up the repo and connecting to the database.
- Week 2: Modelling
- Week 3: Backend deployment; Frontend; LinkedIn writing

Late Submission Policy: A penalty of **10 points per day** will be applied to submissions received after the Final Submission Deadline, unless a formal extension is requested and approved before the deadline.

8. Tips for Success

- Agree on your two modelling tasks on Day 1. Changing direction in Week 2 is very costly.
- Connect to the SQLite database programmatically using Python (sqlite3 or sqlalchemy + pandas). Do not export to CSV manually.
- Save your trained model on Day 1 of Week 2 — even an untrained placeholder — so the backend and frontend can be developed in parallel.
- Test the frontend against the deployed backend before submission, not just locally.
- The LinkedIn post takes longer to write well than expected. Draft it in Week 2 so it only needs polishing in Week 3.
- Commit to GitHub daily with meaningful commit messages. A rich commit history demonstrates genuine teamwork.

9. Academic Integrity

All submitted work must be original and produced by the team. AI tools (e.g. ChatGPT, GitHub Copilot) are permitted as coding assistants, but every member must be able to explain their code and analysis if asked.

Copying code or analyses from other teams will result in disqualification. Submitting work produced by someone outside the team has the same consequence.

10. Submission Instructions

Submit all of the following via the designated form bncc.in/LnTCamp2026FinalProjectSubmission by the Final Submission Deadline:

1. Link to the GitHub monorepo (must be public or shared with the instructor).
2. Link to the deployed frontend application.
3. Backend and API Key (If used).
4. Link to the LinkedIn post.

Checklist before submitting:

- ✓ Notebook runs top-to-bottom without errors

-
- ✓ Model files saved in model/
 - ✓ Backend starts and endpoints respond
 - ✓ Frontend is publicly accessible
 - ✓ LinkedIn post is set to Public
 - ✓ README covers setup for both backend and frontend